

AIM- To interface an HC-SR04 ultrasonic sensor with STM32F446RE and classify distance ranges using visual indication through LEDs.

APPARATUS- STM32 Nucleo-F446RE development board, HC-SR04 ultrasonic sensor, jumper cables, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

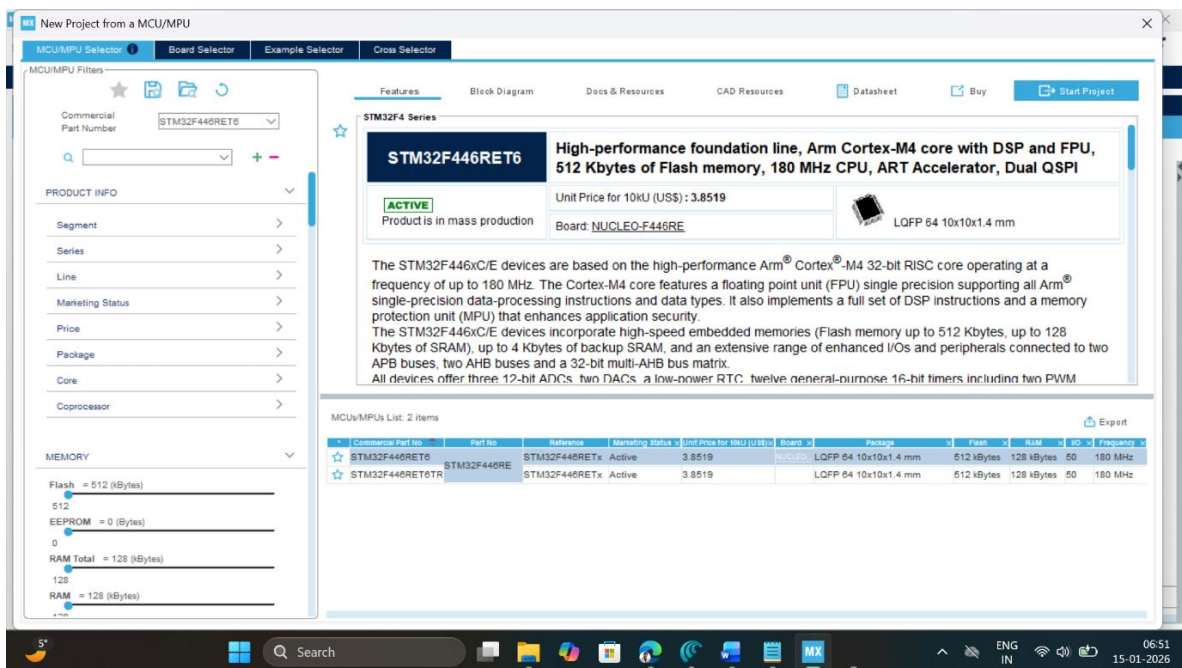
STM32F446RE provides multiple general-purpose I/O (GPIO) ports (**GPIOA–GPIOH**) that can be configured as input, output, alternate function, or analog via GPIO control registers or using the HAL library. When interfacing an HC-SR04 ultrasonic distance sensor with the **STM32F446RE (ARM Cortex-M4)**, specific microcontroller peripherals are used to translate the sensor's ultrasonic time-of-flight signals into meaningful distance data. Typically, one GPIO pin is configured as a digital output to generate the 10 μ s trigger pulse, while another pin is configured as either a GPIO input or a timer input-capture channel to record the width of the echo pulse, which encodes the round-trip travel time of the ultrasonic wave. To calculate the distance, the STM32 uses a **General Purpose Timer (GPT)** configured in **Input Capture mode**. By capturing the timer counts at the rising and falling edges of the Echo pulse, the duration Δt is determined. The distance is then calculated using the speed of sound:

$$d = (v * \Delta t) / 2$$

where v is approximately 343 m/s.

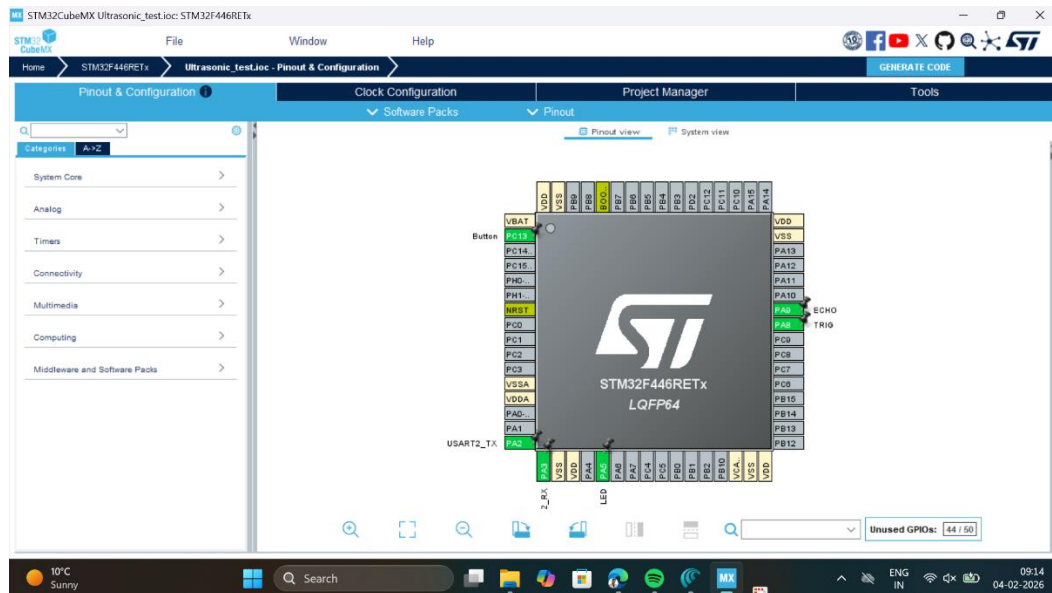
PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.

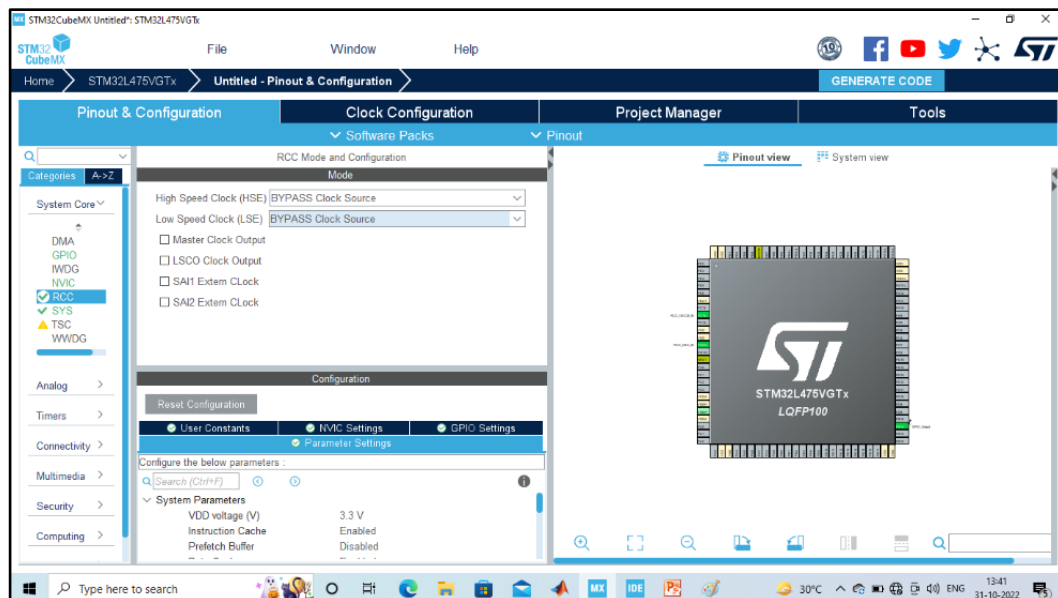


2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.

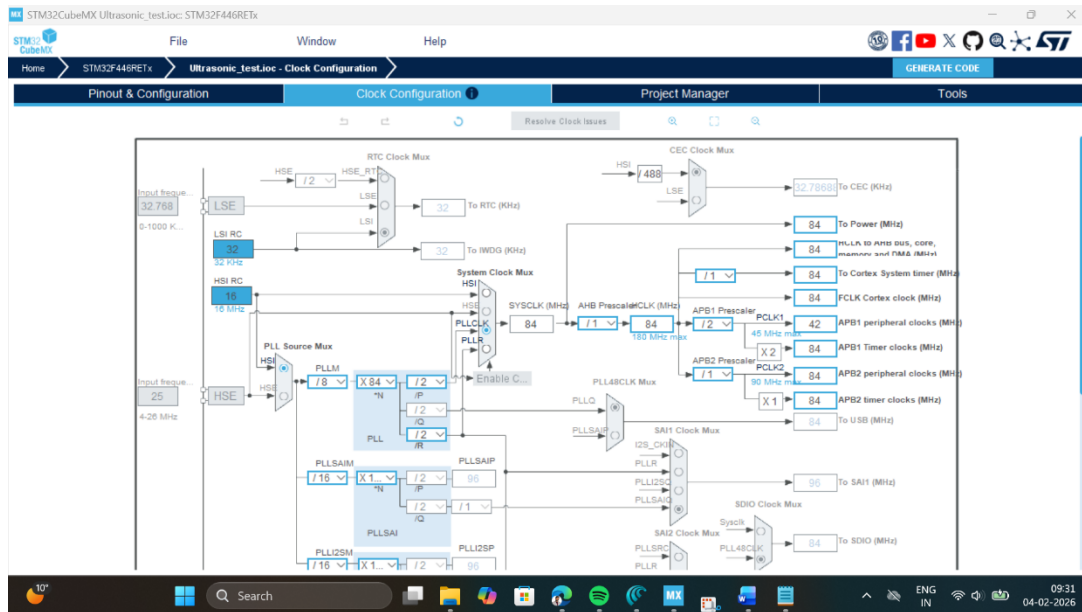
- In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) and PC13 as Pushbutton (USER) relates to it in the board. Furthermore, initialize PA9 (ECHO) as GPIO input, PA9 (TRIG) as GPIO output. Initialize USART 2 by selecting PA2 and PA3 pins as USART_Tx and USART_Rx pins for asynchronous mode.



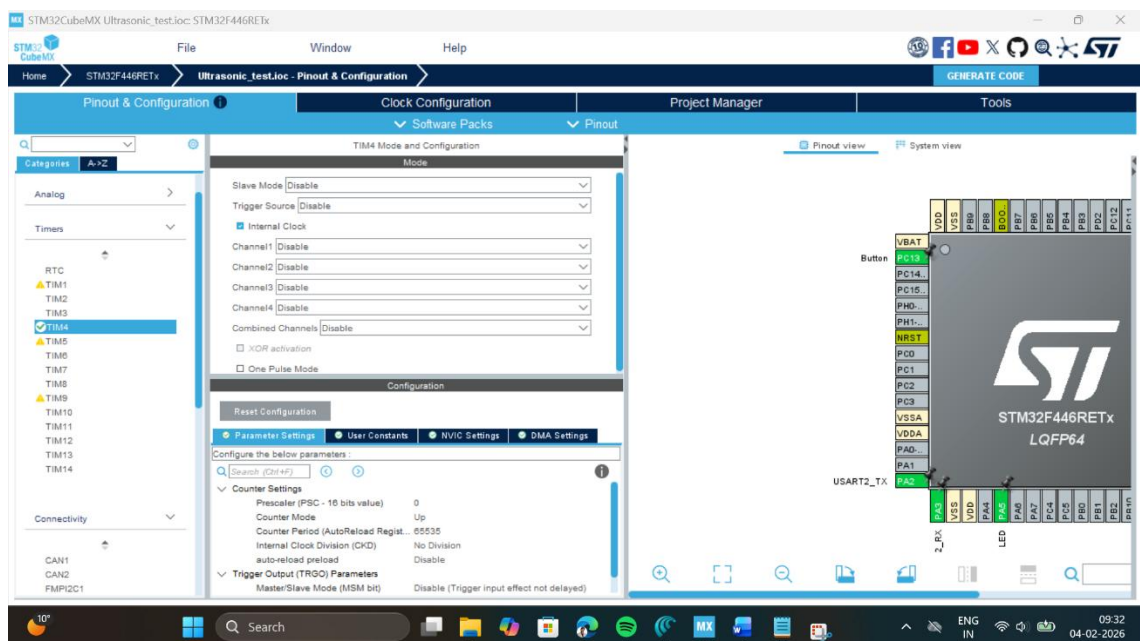
- Clock Selection:** Select the internal clock by clicking RCC and selecting BYPASS clock source



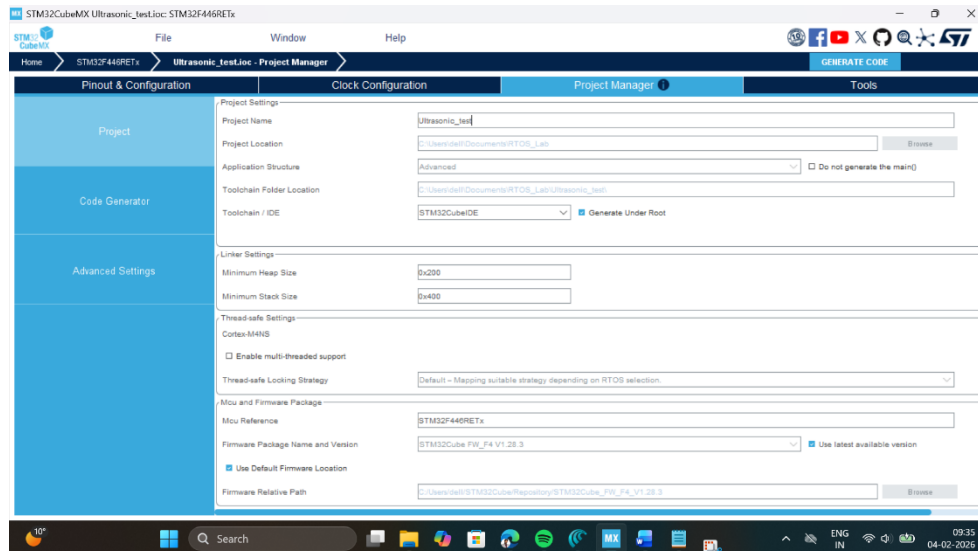
- Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



6. Initialize general purpose time 4 TIM4 from timer section of project configuration and set it to internal clock. Set the Pre-scaler value to 84-1 so that timer can generate timing for 1us from the 84MHz of clock frequency signal.



7. Write Project name and select IDE.



8. Click on generate code.
9. Click on open project.
10. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
11. Open the main source code on Cube IDE by clicking on **main.c**
12. To generate bin file and hex file, right click on project and click properties.
13. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close".
14. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```

/* Private includes ----- */
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */

/* Private typedef ----- */
/* USER CODE BEGIN PTD */
#define usTIM TIM4
/* USER CODE END PTD */

/* USER CODE BEGIN PFP */
void usDelay(uint32_t uSec);

/* USER CODE END PFP */

/* Private user code ----- */
/* USER CODE BEGIN 0 */
//Speed of sound in cm/usec
const float speedOfSound = 0.0343/2;
float distance;
char uartBuf[100];
/* USER CODE END 0 */

```

```
int main(void)
{
    /* USER CODE BEGIN 1 */
        uint32_t numTicks = 0;
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
            //Set TRIG to LOW for few uSec
            HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_RESET);
            usDelay(3);
            /*** START Ultrasonic measure routine ***/
            //1. Output 10 uSec TRIG
            HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_SET);
            usDelay(10);
            HAL_GPIO_WritePin(TRIG_GPIO_Port, TRIG_Pin, GPIO_PIN_RESET);

            //2. Wait for ECHO pin rising edge
            while(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin) == GPIO_PIN_RESET);
            //3. Start measuring ECHO pulse width in uSec

            numTicks = 0;
            while(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin) == GPIO_PIN_SET)
            {
                numTicks++;
                usDelay(2); //2.8uSec
            };

            //4. Estimate distance in cm
            distance = (numTicks + 0.0f)*2.8*speedOfSound;

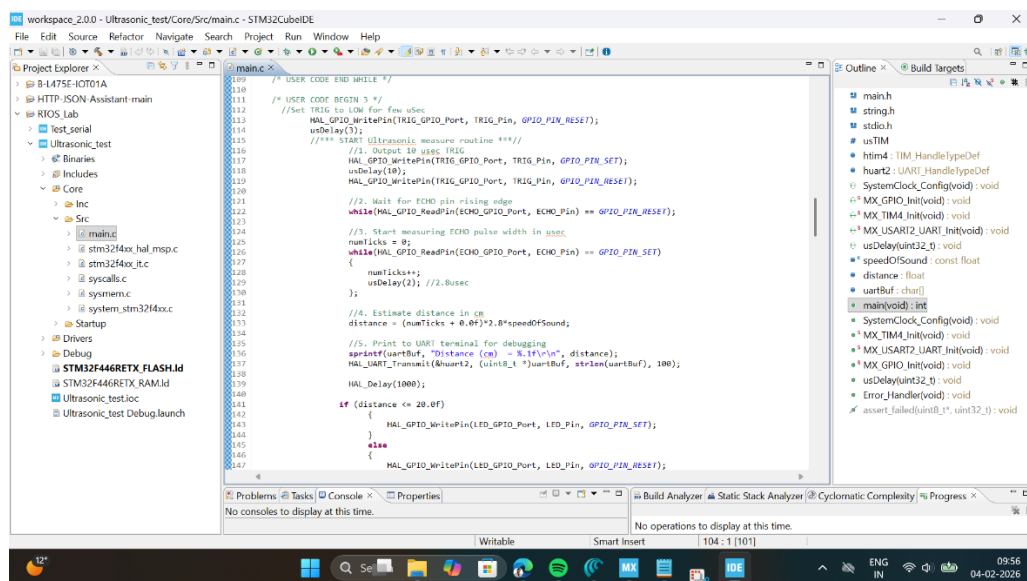
            //5. Print to UART terminal for debugging
            sprintf(uartBuf, "Distance (cm) = %.1f\r\n", distance);
            HAL_UART_Transmit(&huart2, (uint8_t *)uartBuf, strlen(uartBuf), 100);
            HAL_Delay(1000);
            if (distance <= 20.0f)
            {
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
            }
            else
            {
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
            }
        }
    }
}
```

```

/* USER CODE BEGIN 4 */
void usDelay(uint32_t uSec)
{
    if(uSec < 2) uSec = 2;
    usTIM->ARR = uSec - 1;    /*sets the value in the auto-reload register*/
    usTIM->EGR = 1;           /*Re-initialises the timer*/
    usTIM->SR &= ~1;           //Resets the flag
    usTIM->CR1 |= 1;           //Enables the counter
    while((usTIM->SR & 0x0001) != 1);
    usTIM->SR &= ~(0x0001);
}

/* USER CODE END 4 */

```



15. To run this program, connect the board with USB cable, click on build, click 'ok' and click 'switch' on popups.
16. Now you are in debugging mode, click 'Resume' to execute the program.

OBSERVATION:

S. No	Object Distance Actual (in cm)	Distance Measured o UART (in cm)	LED Status (On/OFF)	Echo Pulse width (μs)	Remarks
1	5	5.2	ON	approx 300	Object detected at very close range
2	10	10.1	ON	Approx 570	Object detected accurately
3	15	15.4	ON	Approx 860	System operating normally
4	20	20.3	ON	Approx 1150	Object near the preset limit
5	25	25.8	OFF	Approx 1450	Object is far

RESULT

The HC-SR04 ultrasonic sensor was successfully interfaced with the STM32F446RE microcontroller using suitable GPIO pins for the TRIG and ECHO signals along with an on-chip timer configured for microsecond-level timing. The setup enabled accurate distance measurement over a practical range, with the calculated distance values observed on a serial/UART terminal for verification. An onboard LED was used as a visual indicator to reflect a predefined distance threshold, turning ON when the measured distance was below or equal to the set limit and turning OFF when it exceeded this value. Distance computation was performed using the time-of-flight relation. The distance is then calculated using the speed of sound: $d = (v * \Delta t) / 2$, where v is approximately 343 m/s.

Reflection Summary

1.What is the significance of configuring TIM4 with a prescaler value of 84-1 when the system clock is 84 MHz?

Ans - The STM32 runs at a high speed of 84 MHz, which is very fast to measure small time delays. By setting the prescaler to 84-1, the timer frequency becomes 1 MHz. This means one timer count equals 1 microsecond. It helps in easily measuring the echo pulse width of the ultrasonic sensor.

2. Explain how the usDelay() function works using TIM4 registers (ARR, EGR, SR, CR1). Why is direct timer register manipulation used instead of HAL_Delay()?

Ans -The usDelay() function uses TIM4 registers to generate accurate microsecond delays. The delay value is loaded into the auto-reload register, the timer is started, and the program waits until the update flag is set.

HAL_Delay() is not used because it gives delay in milliseconds, which is not suitable for ultrasonic sensors that need microsecond precision.

3. Why is the usDelay(2) function used inside the while loop that measures the echo pulse width? How does this affect the measurement accuracy?

Ans -The usDelay(2) function is used to add a small fixed delay while the echo pin remains high. Each loop represents a small time interval, and by counting these loops, the total echo pulse width can be calculated. This method gives reasonable accuracy for distance measurement.

4. What is the purpose of the line if(uSec < 2) uSec = 2; in the usDelay() function? What problem does it prevent?

Ans -This condition ensures that the delay value is not too small. Very small delay values may not work properly with the timer and can cause incorrect results. Setting a minimum delay of 2 microseconds makes the delay stable and reliable.

5. How would you modify this experiment to implement an obstacle avoidance system for a mobile robot?

Ans - The distance measured by the ultrasonic sensor can be used to detect obstacles. If the distance is less than a fixed limit, the robot can stop or change direction. If the distance is greater than the limit, the robot can continue moving forward. This way, the system can be used for basic obstacle avoidance.

